Database Systems I / Information Systems I
SQL — Complementary Material

Christian Rauch

(changed: 2026-04-28)

Basic SELECT Structure

Every SQL query follows the same skeleton:

`SELECT` *columns* `FROM` *table(s)* `WHERE` *condition*

- **SELECT** – choose which columns (or expressions) to return
- **FROM** – name the table(s) to read from
- **WHERE** – filter rows *before* grouping

```sql
-- Names of artists born before 1900
SELECT name
FROM   Artists
WHERE  birthDate < '1900-01-01';
```

Filtering with WHERE

- Comparison operators: =, <>, <, >, <=, >=
- Logical operators: `AND`, `OR`, `NOT`
- Set membership: `IN (...)`, `NOT IN (...)`
- Pattern matching: `LIKE '%pattern%'`
- NULL test: `IS NULL`, `IS NOT NULL`

```sql
-- Artifacts worth more than 100,000
SELECT id, title, value
FROM   Artifacts
WHERE  value > 100000;

-- Artists whose name contains 'Monet'
SELECT id, name
FROM   Artists
WHERE  name LIKE '%Monet%';
```

Aggregate Functions

Aggregate functions collapse many rows into a single value.

Function	Meaning
`COUNT(*)`	number of rows
`COUNT(DISTINCT col)`	distinct non-NULL values
`SUM(col)`	total
`AVG(col)`	arithmetic mean
`MIN(col)` / `MAX(col)`	extremes

```sql
-- Total and average artifact value per
    collection
SELECT    collectionTitle,
          COUNT(*)     AS numArtifacts,
          SUM(value)   AS totalValue,
          AVG(value)   AS avgValue
FROM      Artifacts
WHERE     collectionTitle IS NOT NULL
GROUP BY collectionTitle;
```

GROUP BY and HAVING

Rule

Every column in `SELECT` must either be in `GROUP BY` or wrapped in an aggregate function.

- **GROUP BY** – partition rows into groups
- **HAVING** – filter *groups* (applied after aggregation)

```sql
-- Artists with more than one artifact in the database
SELECT    artistId, COUNT(*) AS numArtifacts
FROM      Artifacts
GROUP BY  artistId
HAVING    COUNT(*) > 1;

-- Total advertisement duration per medium
SELECT    mediumName, SUM(duration) AS totalDays
FROM      Advertisements
GROUP BY  mediumName;
```

Join Types

Type	Returns
`(INNER) JOIN`	only rows with a match in *both* tables
`LEFT JOIN`	all rows from the left table; NULLs for unmatched right rows
`RIGHT JOIN`	all rows from the right table; NULLs for unmatched left rows
`FULL JOIN`	all rows from both tables

```sql
-- Artifact titles together with their artist's name
SELECT a.title, ar.name AS artistName
FROM   Artifacts a
JOIN   Artists ar ON a.artistId = ar.id;
```

Self-Join

A table can be joined with *itself* using two aliases. Useful for finding pairs within the same set.

```sql
-- Pairs of artists born in the same year
SELECT ar1.name AS artist1,
       ar2.name AS artist2,
       YEAR(ar1.birthDate) AS birthYear
FROM   Artists ar1
JOIN   Artists ar2
       ON  YEAR(ar1.birthDate) = YEAR(ar2.
           birthDate)
       AND ar1.id < ar2.id;  -- avoids (A,B) and
            (B,A)
```

The condition `ar1.id < ar2.id` removes both duplicates and self-pairs in one step.

Subqueries

A *subquery* is a SELECT nested inside another statement.

Scalar returns a single value; used with =, >, . . .

Row/Table returns a set; used with IN, NOT IN, EXISTS

Derived table used in FROM with an alias

```
1  -- Scalar subquery: most expensive artifact
2  SELECT title, value
3  FROM   Artifacts
4  WHERE  value = (SELECT MAX(value) FROM Artifacts
       );
5
6  -- IN subquery: artifacts in a specific
       collection topic
7  SELECT id, title
8  FROM   Artifacts
9  WHERE  collectionTitle IN (
10     SELECT title FROM Collections WHERE topic =
           'painting'
11 );
```

Correlated vs. Uncorrelated Subqueries

Correlated

References a column from the outer query – re-executed for *every* outer row.

Uncorrelated

Independent of the outer query – executed *once*; result reused.

Both styles produce the same result. The uncorrelated version (derived table or `JOIN`) is usually more efficient because the subquery runs only once.

The next three slides show the same query written both ways.

Example 1: Scalar Comparison (Correlated)

Task: Find artifacts whose value exceeds the average value of all artifacts by the same artist.

Correlated – subquery references outer alias a, so it is re-evaluated once per artifact:

```
SELECT id, title, value, artistId
FROM   Artifacts a
WHERE  value > (
    SELECT AVG(value)
    FROM   Artifacts b
    WHERE  b.artistId = a.artistId   --
        references outer row
);
```

Example 1: Scalar Comparison (Uncorrelated)

Task: same – now rewritten without correlation.

Uncorrelated – a derived table computes all averages *once*, then the outer query joins against it:

```
SELECT a.id, a.title, a.value, a.artistId
FROM   Artifacts a
JOIN  (SELECT artistId, AVG(value) AS avgVal
       FROM   Artifacts
       GROUP BY artistId) avg_a
       ON a.artistId = avg_a.artistId
WHERE  a.value > avg_a.avgVal;
```

Example 2: EXISTS (Correlated)

Task: Find media outlets that have placed at least one advertisement.

Correlated – `EXISTS` references the outer alias `m`, so the subquery is re-evaluated for every medium:

```
SELECT name
FROM   Medium m
WHERE  EXISTS (
    SELECT 1
    FROM   Advertisements a
    WHERE  a.mediumName = m.name   -- references
        outer row
);
```

Example 2: IN (Uncorrelated)

Task: same – now rewritten without correlation.

Uncorrelated – the inner `SELECT` is fully independent; it runs once and produces a set for `IN`:

```
SELECT name
FROM   Medium
WHERE  name IN (
    SELECT DISTINCT mediumName    -- independent
        subquery
    FROM   Advertisements
);
```

Example 3: NOT EXISTS (Correlated)

Task: Find artists who have no artifact in the database.

Correlated – `NOT EXISTS` references the outer alias `ar`, re-evaluated for every artist:

```
1  SELECT id, name
2  FROM   Artists ar
3  WHERE  NOT EXISTS (
4      SELECT 1
5      FROM   Artifacts a
6      WHERE  a.artistId = ar.id   -- references
           outer row
7  );
```

Example 3: NOT IN (Uncorrelated)

Task: same – now rewritten without correlation.

Uncorrelated – the subquery builds the full set of artist IDs once; `NOT IN` filters against it:

```
1  SELECT id, name
2  FROM   Artists
3  WHERE  id NOT IN (
4      SELECT DISTINCT artistId     -- independent
           subquery
5      FROM   Artifacts
6      WHERE  artistId IS NOT NULL
7  );
```

Note: the `IS NOT NULL` guard is necessary – `NOT IN` returns no rows at all if the subquery contains any `NULL` value.

EXISTS

`EXISTS` returns `TRUE` if the subquery returns at least one row. Naturally correlated; stops as soon as one match is found.

```
1  -- Collections that contain at least one
       painting
2  SELECT title
3  FROM   Collections c
4  WHERE  EXISTS (
5      SELECT 1
6      FROM   Artifacts a
7      JOIN   ArtifactPaintings ap ON a.id = ap.id
8      WHERE  a.collectionTitle = c.title
9  );
```

`SELECT 1` (or `SELECT *`) is conventional inside `EXISTS` – the actual column values are irrelevant.

INSERT

```sql
-- Single row
INSERT INTO Artists (id, name, birthDate,
    deathDate, bio)
VALUES (6, 'Pablo Picasso', '1881-10-25', '
    1973-04-08', NULL);

-- Insert from a SELECT (copy a subset of rows)
INSERT INTO ExhibitedAt (artifactId,
    exhibitionTitle)
SELECT a.id, 'Modern Visions'
FROM   Artifacts a
JOIN   ArtifactSculptures s ON a.id = s.id
WHERE  a.collectionTitle = 'Modern Art';
```

The `INSERT ...SELECT` pattern copies data without leaving the database.

UPDATE

```sql
-- Set a placeholder description for artifacts
    with none
UPDATE Artifacts
SET    description = 'No description available'
WHERE  description IS NULL;

-- Move all exhibitions from room '01-100' to
    '01-150'
UPDATE Exhibitions
SET    room = '01-150'
WHERE  room = '01-100';
```

DELETE

```sql
-- Remove advertisements with zero duration
DELETE FROM Advertisements
WHERE   duration = 0;
```

Caution

`DELETE` without a `WHERE` clause removes *all* rows.

Always test with `SELECT` first.

Handling NULL

- `NULL` means *unknown* – not zero, not empty string.
- Any arithmetic or comparison with `NULL` yields `NULL`.
- Test with `IS NULL` / `IS NOT NULL`, not with `= NULL`.
- `COUNT(*)` counts all rows; `COUNT(col)` ignores NULLs.
- `LEFT JOIN` produces NULLs for unmatched rows – useful for "find rows with no match" patterns.

```
1  -- Artifacts not yet assigned to any collection
2  SELECT id, title
3  FROM   Artifacts
4  WHERE  collectionTitle IS NULL;
5
6  -- Artists with no biographical information
7  SELECT id, name FROM Artists WHERE bio IS NULL;
```

ALTER TABLE – Adding and Removing Columns

```sql
-- Add a new nullable column
ALTER TABLE Artifacts
  ADD COLUMN location VARCHAR(100);

-- Add a column with a default value
ALTER TABLE Exhibitions
  ADD COLUMN capacity INT DEFAULT 0;

-- Drop a column
ALTER TABLE Artists
  DROP COLUMN bio;
```

Caution

`DROP COLUMN` permanently destroys the data in that column.

ALTER TABLE – Modifying Columns

```sql
-- Change the data type of a column
ALTER TABLE Artifacts
  MODIFY COLUMN year INT;

-- Rename a column (and optionally change its
    type)
ALTER TABLE Artists
  CHANGE COLUMN name fullName VARCHAR(100);

-- Give a column a new default value
ALTER TABLE Exhibitions
  ALTER COLUMN room SET DEFAULT '01-100';

-- Remove a default value
ALTER TABLE Exhibitions
  ALTER COLUMN room DROP DEFAULT;
```

ALTER TABLE – Constraints

```sql
-- Add a FOREIGN KEY constraint
ALTER TABLE Artifacts
  ADD CONSTRAINT fk_artist
  FOREIGN KEY (artistId) REFERENCES Artists(id);

-- Add a UNIQUE constraint
ALTER TABLE Medium
  ADD CONSTRAINT uq_contact UNIQUE (contact);

-- Drop a constraint by name
ALTER TABLE Artifacts
  DROP FOREIGN KEY fk_artist;

-- Rename the table itself
ALTER TABLE ArtifactPaintings
  RENAME TO Paintings;
```

Date Functions

Function	Returns
`CURDATE()`	today's date
`NOW()`	date + time
`YEAR(d)`	year part
`MONTH(d)`	month part
`DAY(d)`	day part
`DATEDIFF(a,b)`	days $a - b$

```
1  -- Exhibitions starting
2  -- in the current year
3  SELECT title, startDate
4  FROM   Exhibitions
5  WHERE  YEAR(startDate)
6         = YEAR(CURDATE());
7
8  -- Days running so far
9  SELECT title,
10   DATEDIFF(CURDATE(),
11            startDate)
12     AS daysRunning
13 FROM   Exhibitions
14 WHERE  startDate <=
     CURDATE();
```

String Functions

Function	Effect
`UPPER(s)` / `LOWER(s)`	change case
`LENGTH(s)`	number of characters
`TRIM(s)`	remove leading/trailing spaces
`SUBSTRING(s, pos, len)`	extract substring
`CONCAT(s1, s2, ...)`	concatenate strings
`REPLACE(s, old, new)`	substitute substring

```sql
-- Artists whose name has more than 15
    characters
SELECT name, LENGTH(name) AS nameLen
FROM   Artists
WHERE  LENGTH(name) > 15;

-- Display collection titles in upper case
SELECT UPPER(title) AS titleUpper
FROM   Collections;
```

Pattern Matching with LIKE

`LIKE` matches strings using two wildcards:

Wildcard	Matches	Example
`%`	any sequence of characters (incl. empty)	`'Van%'`
`_`	exactly one character	`'_onet'`

```sql
-- Names starting with 'Van'
SELECT name FROM Artists WHERE name LIKE 'Van%';

-- Names where the second character is 'o' (e.g. 'Monet', 'Rodin')
SELECT name FROM Artists WHERE name LIKE '_o%';

-- Titles containing the word 'Night' anywhere
SELECT title FROM Artifacts WHERE title LIKE '%Night%';

-- Titles that are exactly 10 characters long
SELECT title FROM Artifacts WHERE title LIKE '
```